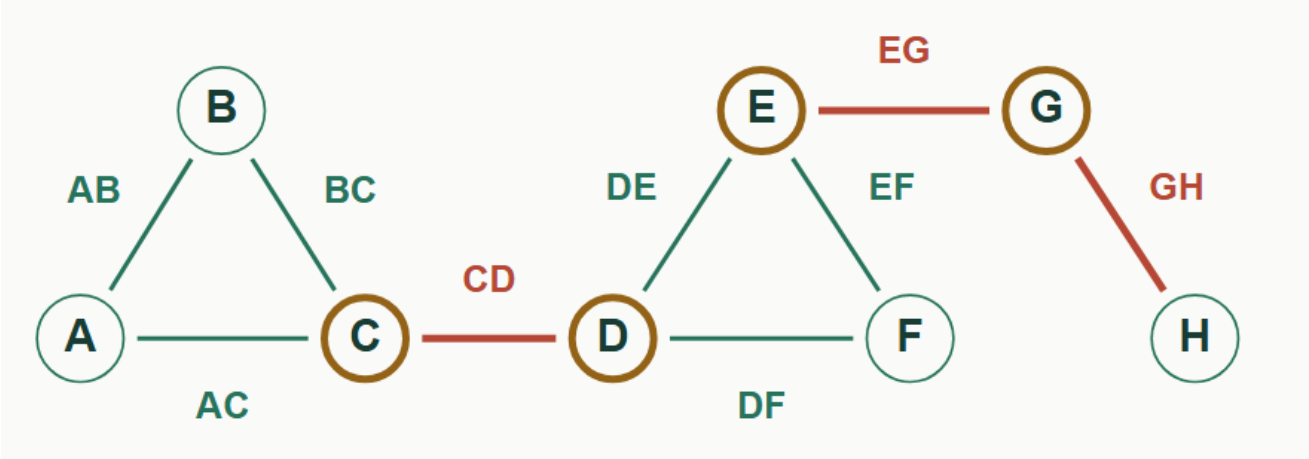
Treasure-Island Bridge Safety Map

Technical project report | Five implementation phases and verified extensions

An interactive graph-analysis tool that helps a festival volunteer identify the connections and islands whose closure would split the network.



Demonstration network: 8 islands, 9 undirected edges and two cycles. Red edges are bridges; amber node outlines mark articulation islands. Node and edge labels are separated from the connecting lines.

Delivered result

The original graph produces **54 deterministic events**, one connected component, bridges **CD, EG, GH**, and articulation islands **C, D, E, G**. A step selects one recorded event and its complete snapshot, keeping the map, numerical table, explanation and summary synchronized.

Scope	Implementation
Core workflow	Load, Step, Run to End and exact original-demo Reset; complete history retained for inspection.
Optional requirements	Gentle replay transitions with reduced-motion support, plus a printable final safety summary.
Further exploration	Selectable graph examples, single bridge/island closure comparison, and reached-proof inspection.

Quality checkpoint: 337 tests in 17 files, production build and lint passed. Phase-specific counts are historical checkpoints; generated graph cases are not additional test counts.

Open the live application | Browse the source repository

Scope, acceptance and delivery plan

The required workflow was implemented before adding the optional presentation and exploration features.

Requirement	Delivered behavior and evidence
Input contract	Engine accepts 1..12 nodes and 0..20 undirected edges. The 8-node, 9-edge demo meets the narrower demonstration size.
Validation	Unknown runtime input is checked completely: structure, IDs, names, finite coordinates, endpoints, self-loops and duplicate unordered pairs. Errors identify fields/items.
Determinism	Ascending ASCII root/neighbor IDs, a global zero-based discovery clock and sorted critical summaries. Coordinates and source ordering cannot affect results.
Explanatory replay	Ten typed event variants record focus, numerical details, proofs and a frozen post-event snapshot. Both adjacency entries are inspected.
Consistent presentation	One selected ReplayView drives the map, table, event panel, reached history and summary. No future classification is shown early.
Failure and Reset	Invalid loads clear stale output atomically. Reset restores the independently owned original demo, including after replacement/error.
Independent verification	Hand-derived reference table, removal-based oracle, event-transition checks and React/browser verification.
Both optionals	Brief reduced-motion-aware transitions and a completed-analysis print report without executing analysis again.

Five phase gates

Phase	Bounded deliverable	Tests at gate
1	Readonly graph models, fresh demo factory and runtime validation	91
2	Sorted adjacency, Tarjan Strategy and injected AnalysisService	149 (+58)
3	Typed events, numerical proofs and independent frozen snapshots	190 (+41)
4	Pure replay reducer/selectors and React controller	261 (+71)
5	SVG map, controls, table, history, explanations and safety summary	293 (+32)

Reliability and both optional features added 10 cases (303 total). Exploration added 34 cases, bringing the current total to **337**. Earlier algorithm assertions were retained.

Sources: implementation plan; phase evidence; optional-feature verification.

Technology and architecture

Calculation and replay remain testable without React; presentation consumes the prepared results.

Area	Choice and purpose
Frontend	React, TypeScript and Vite; native SVG for the graph; Tailwind CSS and project CSS for layout and state styling.
State	React useReducer connects a pure application reducer to the controller. A single owner avoids an additional global state dependency.
Verification	Vitest, Testing Library and file-scoped jsdom; TypeScript/Vite production build; Oxlint; actual Chrome captures and measurements.
Delivery	Git, public GitHub repository, GitHub Actions quality workflow and a Render static site serving dist.

Dependency boundaries

Layer	Responsibility
Domain	Graph contracts, validation, sorted adjacency, GraphAnalyzer, Tarjan calculation, typed replay events and immutable traversal snapshots. No React imports.
Application	AnalysisService orchestrates validation and analysis; pure replay state, reducer and selectors; pure connectivity comparison and proof selection.
Presentation	App is the composition root. useReplayController adapts the injected service and reducer. Panels render one derived ReplayView.

Patterns used deliberately

Strategy: GraphAnalyzer is the contract implemented by TarjanLowLinkAnalyzer. **Constructor dependency injection:** AnalysisService receives the strategy; App supplies the concrete implementation. Tests can inject a controlled analyzer or prepare service.

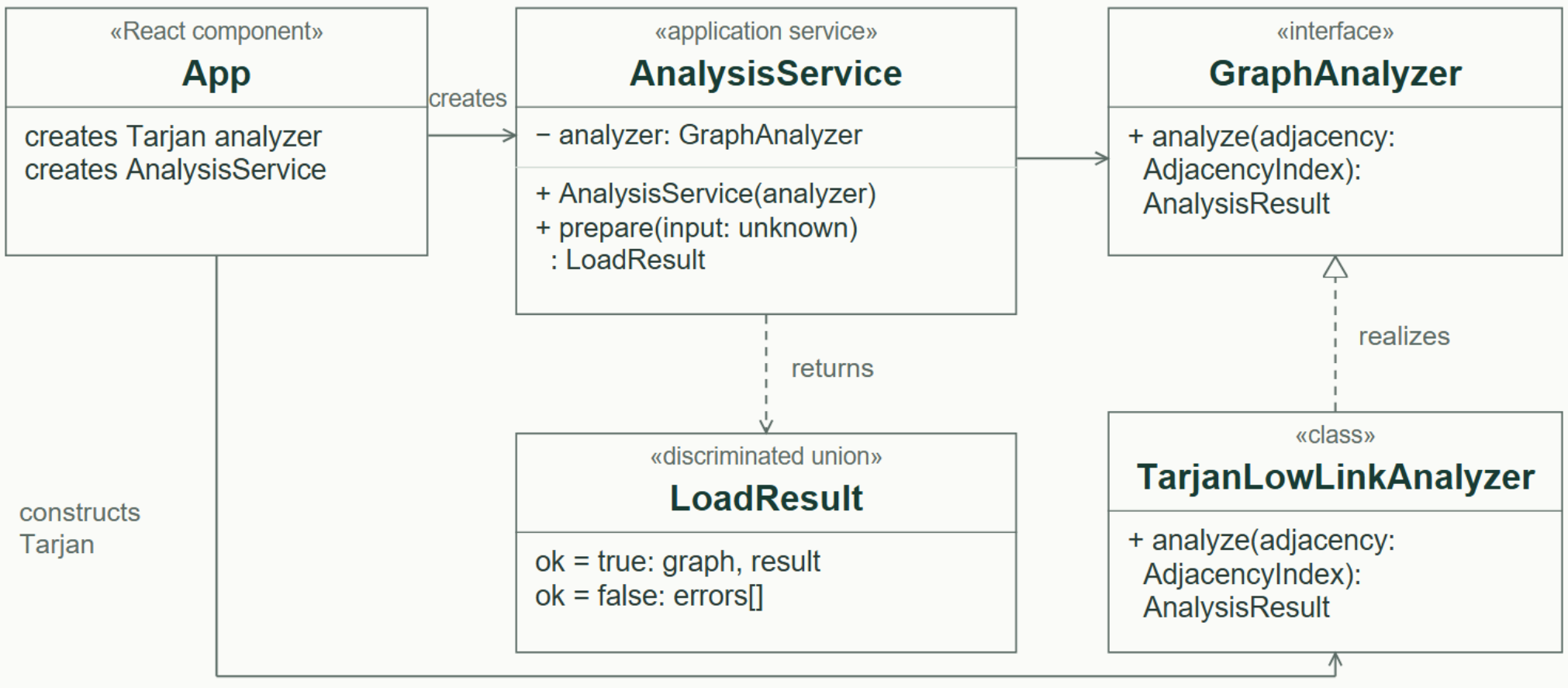
Reducer pattern: typed actions produce predictable transitions over a discriminated state union. **Immutable snapshot history:** recorded values are copied and frozen, so rendering a past event cannot observe later DFS mutations.

Single responsibilities separate validation, calculation, replay and rendering. Dependency inversion occurs at the analyzer boundary; the hook depends only on prepare. These are concrete design choices, not claims of a Factory Method hierarchy, Command objects or durable event sourcing.

Sources: design and diagrams; design-pattern discussion.

UML: analysis and dependency injection

The application service depends on a strategy contract. App chooses and constructs the concrete analyzer.



The diagram uses the implemented names and public types. A solid open arrow denotes a dependency; the dashed line with an unfilled triangle denotes interface realization. The editable version is included in the PowerPoint presentation.

Execution boundary: prepare(input: unknown) validates first. Invalid input returns errors without DFS. Valid input is copied into the accepted graph, converted into sorted adjacency and analyzed once. LoadResult distinguishes success from failure.

Source: architecture and UML sources.

Tarjan analysis and independent reference

The reference is documented and tested separately; production analysis never reads an expected-result table.

DFS visits roots and neighbors by ascending ID. A newly discovered node receives discovery = low = the next global clock value. Returning from a tree child applies **min(low[parent], low[child])**. A visited non-parent neighbor contributes **discovery[neighbor]**, not its low value.

Node	Sorted neighbors	Parent	Discovery	Final low
A	B, C	-	0	0
B	A, C	A	1	0
C	A, B, D	B	2	0
D	C, E, F	C	3	3
E	D, F, G	D	4	3
F	D, E	E	5	3
G	E, H	E	6	6
H	G	G	7	7

Why the equality case matters

Reached event	Recorded proof and interpretation
21: return F to E	low[E] changes from 4 to 3: min(4, low[F] = 3) = 3. F can reach ancestor D.
39: classify DE	low[E] > discovery[D] is 3 > 3, which is false. DE has an alternate route through F and is not a bridge.
40: articulate D	For non-root D, low[E] >= discovery[D] is 3 >= 3, which is true. Removing D separates the child region from C and the earlier network.

Root rule: at least two DFS-tree children, rather than graph degree. Root A has neighbors B and C but only one tree child, because C is reached through B. A is not an articulation island.

Final result: one component; bridges CD, EG, GH; articulation islands C, D, E, G. The cycles A-B-C and D-E-F preserve their internal alternate routes.

Source: independent reference graph.

Events, snapshots and predictable replay

The prepared history is the source of replay truth; navigating it never reruns DFS.

Each event carries a deterministic zero-based index, a discriminated type, node/edge focus, a readable message, a typed detail payload and the complete snapshot immediately after the event. Visible event numbers add one to the domain index.

Event family	Recorded behavior
COMPONENT_START / COMPONENT_END	Bound a component in sorted-root order; component progress is visible only when reached.
DISCOVER / FINISH_NODE	Record discovery/low initialization and node completion. Evaluate the root child-count rule before FINISH_NODE.
INSPECT_TREE / INSPECT_PARENT / INSPECT_VISITED	Inspect each undirected edge from both adjacency entries, including the parent return entry and visited-neighbor contribution.
RETURN_CHILD / CLASSIFY_EDGE / ARTICULATION	After child completion: RETURN_CHILD, then CLASSIFY_EDGE, then any newly detected ARTICULATION. Unchanged low values and negative classification outcomes are retained. Articulation markers are deduplicated.

Ownership: every captured record and nested array is copied and deeply frozen at runtime. The initial snapshot has unvisited nodes, unset discovery/low values, uninspected edges, zero components and empty summaries. finalSnapshot is the last event's snapshot by identity.

Load and replay sequence

Load: control calls hook.load; the hook calls service.prepare; validation either returns structured errors or passes a graph to adjacency construction and the injected analyzer. The hook dispatches LOAD_SUCCESS or LOAD_FAILURE. The selector derives one view for all panels.

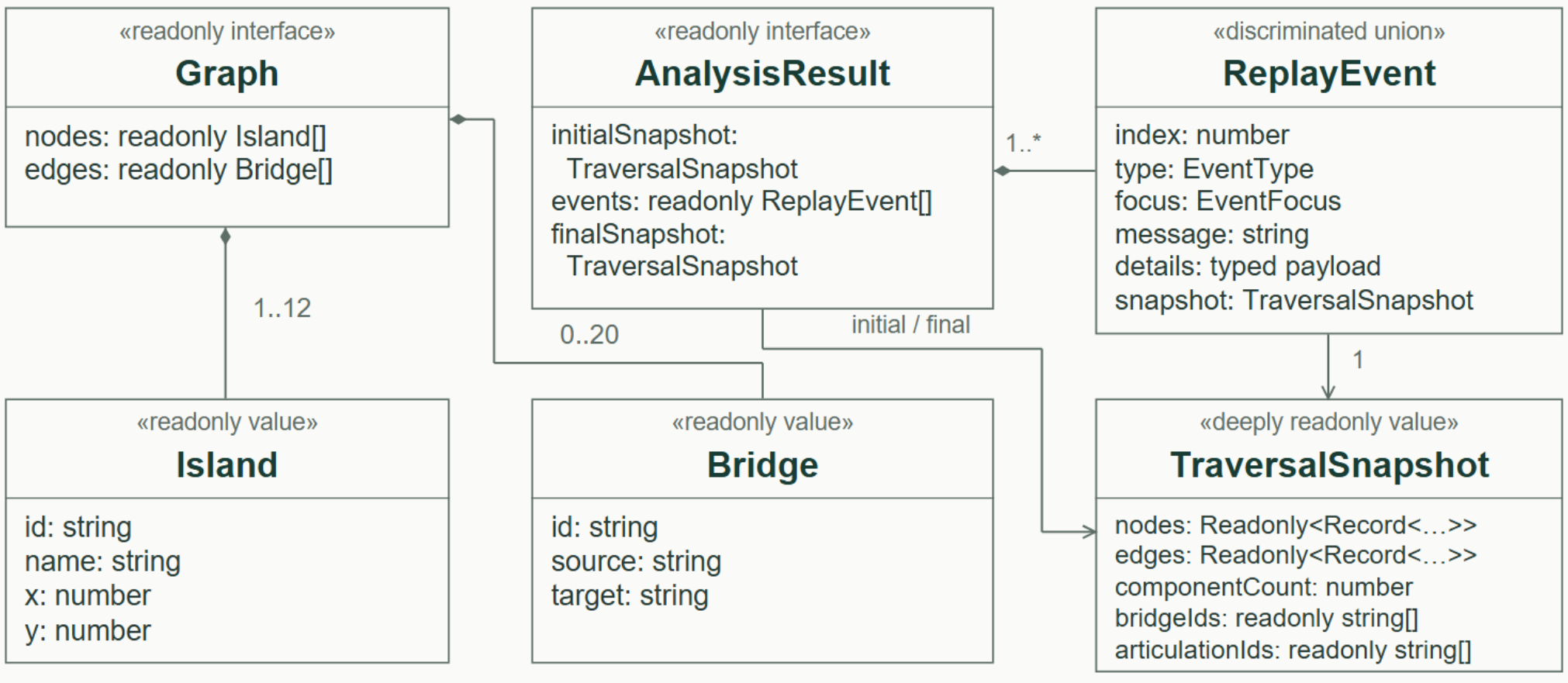
Step / Run / Reset: controls dispatch reducer actions directly. Step advances one event; Run selects the final prepared event; Reset restores the saved original preview. Reducer and selectors do not validate or execute DFS.

Status	Visible selection
EMPTY / ERROR	Original unvisited preview / no current graph or analysis, with identifiable errors. Step and Run are harmless.
READY	Cursor -1 selects initialSnapshot, with no event or reached history.
RUNNING / COMPLETED	Selected event snapshot and the reached history prefix / final snapshot with the complete retained history. Completion is clamped.

Sources: event schema; replay contract and sequences.

UML: graph and replay value models

Readonly contracts describe accepted input and historical state. Runtime freezing provides the ownership guarantee.



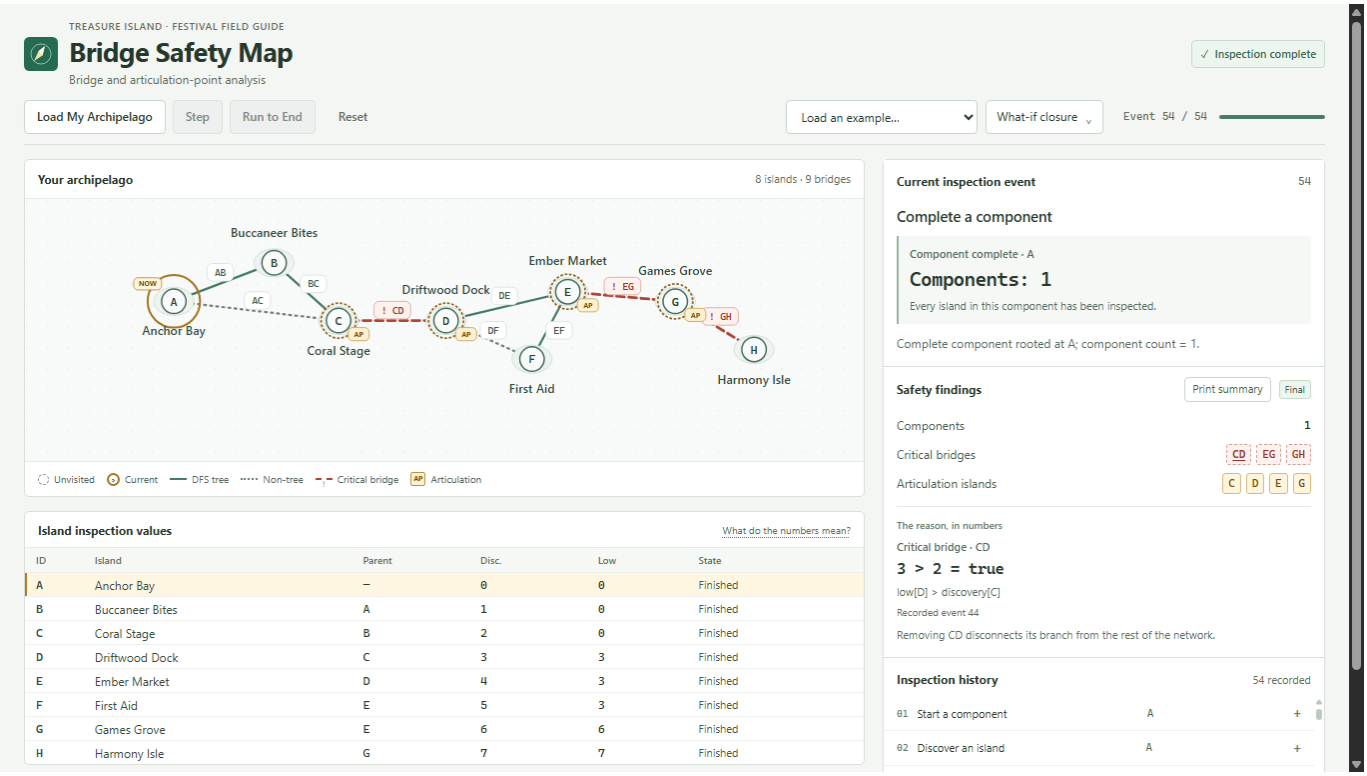
The graph owns node and edge collections. AnalysisResult contains the prepared event sequence and initial/final snapshots; every event selects exactly one post-event snapshot. The diagram abbreviates node/edge record fields for readability.

Selector contract: one ReplayView exposes the current graph, selected snapshot, current event/focus, reached history, status/progress, errors and control availability. Full history remains internal until reached; final values cannot leak into earlier views.

Source: event and snapshot schema.

Interface, optional features and exploration

Actual production capture. Styling expresses traversal, focus and criticality without relying on color alone.



One consistent inspection view

The SVG map, numerical table, current proof, history and safety summary all consume the same selected snapshot. Native buttons, labels, symbols, line styles and accessible descriptions support inspection. Desktop and narrow mobile layouts were checked in Chrome.

Both optional requirements

Brief event/focus transitions respect reduced motion and never advance the cursor. Print summary uses the completed selected view and recorded proofs; the demo produces one A4 page. Printing does not analyze again or change the cursor.

Exploration beyond the baseline

Cycle, branching-root and disconnected/isolated examples follow the existing load path. Closing CD gives 1 to 2 components; closing DE keeps 1; closing D gives 1 to 2. A separate pure flood fill shows surviving partitions; it does not reclassify the remaining graph or alter the baseline history/table/report.

Sources: UI evidence; optional features; exploration evidence.

Verification and AI-assisted development

AI output was constrained by explicit contracts and checked against independently derived evidence.

What the test evidence establishes

The current suite has **337 passing tests in 17 files**. Domain and application suites run without a browser; React tests use file-scoped jsdom. Hosted GitHub Actions also passed locked installation, all tests, production build and lint on Ubuntu.

Check	Coverage
Independent numerical oracle	Removal-based flood fill over all 1,099 simple graphs on node-set sizes 1..5. This is inside automated cases, not 1,099 additional Vitest tests.
History correctness	Exact trace, twice-per-edge inspections, numerical old/new lows, ordering, root/equality rules, frozen snapshots, shuffled inputs, coordinate independence and disconnected order. Event transitions are checked over 75 small graphs.
Replay and integration	One analysis per Load; no DFS for Step/Run/Reset/rerenders; StrictMode; repeated histories; valid-invalid-valid recovery; two different invalid loads clear stale results.
Real browser	Chrome desktop 1366 x 768, mobile widths 390 and 320, print media, reduced motion, keyboard operations and closure/proof inspection. Recorded page/console errors: zero.

Tools, prompts and real iterations

The OpenAI assistant in chat and Codex desktop supported planning, implementation, test authoring, documentation and diagnosis. Work was split into bounded phases with explicit preserved contracts and verification gates.

An actual Phase 3 instruction was: *"Generate events during the existing DFS traversal."* Another was: *"Preserve all Phase 2 result fields and numerical behavior."* These constrained how replay was added without replacing the accepted calculation.

The development record documents runtime validation refinements, accessible naming, map-label spacing and a slow 54-event integration test. The test repair cached/scoped queries, limited workers to two and applied a 20-second budget only to the two exhaustive replay cases. All assertions were retained; other cases keep the five-second default.

Actual Phase 3-5 requests are saved separately from the curated engineering prompt collection. The collection is supporting guidance, not a claimed verbatim transcript of every historical chat.

Sources: AI development record; actual Phase 3 request; current verification.

Delivery, tradeoffs and next modification

Verified application checkpoint: 88476a3. This report and the corrected presentation are documentation-only additions.

Run and inspect

Live application
Source repository
Hosted checks for the application checkpoint

```
git clone https://github.com/sonisiiuu7/treasure-island-map.git
cd treasure-island-map
npm.cmd ci
npm.cmd run dev -- --host 127.0.0.1 --port 5175 --strictPort
```

Windows commands above work after Git and a supported Node.js installation are available. In PowerShell, Set-Location is supported; in Command Prompt, use cd /d. Check with npm.cmd run test:run, npm.cmd run build and npm.cmd run lint.

GitHub Actions runs installation, tests, build and lint. Render hosts the static dist output and is configured to deploy main commits. The repository is public with the owner's authorization; no backend, database or application secrets are needed.

Cost and practical limits

The DFS core is **O(V + E)**, with additional sorting. For K events, complete snapshots require **O(K(V + E))** copied data, plus sorting/freezing work. K grows with traversal, so the full history can be quadratic overall. This is a deliberate clarity tradeoff for at most 12 nodes and 20 edges. Larger graphs would motivate iterative DFS and delta/checkpoint history.

Evidence covers Chrome with mobile viewport emulation. It does not claim physical-device, cross-browser, screen-reader, physical-printer or arbitrary-coordinate label-collision certification. Closure simulation compares connectivity only and does not compute a new critical set.

A bounded live modification

A history filter is a proposed exercise, not an implemented feature: filter only the already-replayed history, retain the selected event and cursor, and leave analysis/snapshot ownership unchanged. Verify prefix boundaries, Reset/error clearing and analysis-call counts with focused selector/UI tests, then run the regression gates.

Source and requirement index

Requirements were read from the supplied *Student_SPR26_D2_P01-treasure-island-bridge-safety-map* and *STUDENT_GUIDE (1)* documents. Implementation claims are supported by the repository evidence linked throughout this report.

Design, UML and sequence diagrams | Implementation plan
Reference table | Demo walkthrough
Deployment record | Presentation PDF